

CODING STANDARD DOCUMENT

Planet Motor BMW

Standar Penulisan Kode untuk Frontend, Backend, API, Database, dan Testing

Project	Planet Motor BMW
Tim	Digital Dynasty
Jenis Dokumen	Coding Standard Document
Stack Utama	Next.js, React, TypeScript, Go API, PostgreSQL
Tanggal	19 Mei 2026

1. Pendahuluan

Coding Standard Document ini mendefinisikan aturan penulisan kode untuk project web Planet Motor BMW. Project ini memiliki frontend berbasis Next.js dan React, backend Go API, serta database PostgreSQL. Standar ini dipakai agar implementasi katalog produk, admin inventory, stok, dan integrasi eksternal tetap konsisten.

Dokumen ini tidak berisi credential, URL production, token, password, maupun konfigurasi rahasia. Semua contoh penulisan environment variable hanya berupa nama variabel, bukan nilai production.

Tujuan Standar

- Membuat struktur kode mudah dipahami oleh anggota tim.
- Menjaga konsistensi naming, folder, API contract, dan error handling.
- Mengurangi bug akibat pola implementasi yang berbeda-beda.
- Memastikan code review dan testing memiliki acuan yang jelas.

2. Ruang Lingkup

Area	Standar yang Diatur	Output yang Diharapkan
Frontend	TypeScript, React component, Next.js App Router, styling, state, API client, aksesibilitas.	UI konsisten, responsive, dan mudah dirawat.
Backend	Go package, handler, service, repository, middleware, database transaction, response JSON.	API stabil, jelas, dan aman.
Database	Migration, query, index, naming kolom, audit field, dan data integrity.	Schema mudah dilacak dan tidak merusak data.
Testing	Unit test, API test, build check, dan skenario manual.	Perubahan bisa diverifikasi sebelum deploy.
Security	Secret management, auth cookie, validation, upload, dan CORS.	Data sensitif tidak bocor dan akses admin terlindungi.

3. Prinsip Umum Penulisan Kode

Prinsip	Aturan
Readability	Kode harus mudah dibaca sebelum dianggap pintar. Hindari abstraksi yang tidak memberi manfaat nyata.
Consistency	Ikuti pola folder, nama fungsi, nama type, dan response yang sudah ada di project.
Single Responsibility	Komponen, service, handler, dan repository memiliki tanggung jawab yang jelas.
Explicit Error	Error tidak boleh diabaikan. Semua error penting harus dikembalikan, dicatat, atau ditangani.
No Secret in Code	Credential tidak boleh ditulis di source code, README, test fixture, atau dokumen publik.
Small Change	Perubahan sebaiknya kecil, fokus, dan mudah direview.

4. Standar Frontend

Frontend menggunakan Next.js App Router, React, TypeScript, Tailwind CSS, Zustand, Zod, dan Vitest. Standar frontend difokuskan pada komponen yang reusable, data flow yang jelas, dan tampilan yang konsisten.

4.1 Struktur Folder Frontend

Folder	Fungsi	Standar
src/app	Routing halaman public, auth, admin, dan API route/rewrite.	Gunakan struktur route yang jelas. Hindari business logic besar langsung di page.
src/components	Komponen UI reusable dan section halaman.	Pisahkan komponen shared, public, dan admin bila tanggung jawab berbeda.
src/hooks	Custom hook React.	Nama hook diawali use, misalnya useStockStream.
src/lib	API client, helper, i18n, formatting, auth helper.	Helper harus pure jika memungkinkan dan mudah diuji.
src/stores	Client-side state management.	State global hanya untuk kebutuhan lintas halaman/komponen.
src/types	TypeScript type dan interface.	Type mengikuti shape response API dan domain aktif.

4.2 Naming dan TypeScript

Item	Standar
File component	Gunakan kebab-case, misalnya product-card.tsx dan channel-icon.tsx.
Component	Gunakan PascalCase, misalnya ProductCard dan AdminSidebar.
Hook	Gunakan camelCase dengan prefix use, misalnya useStockStream.
Type/Interface	Gunakan PascalCase, misalnya Product, Category, StockLog.
Variable/function	Gunakan camelCase dan nama yang menjelaskan tujuan.
Any	Hindari any. Gunakan unknown atau type spesifik bila data belum pasti.

Aturan React Component

- Komponen visual kecil sebaiknya menerima props yang jelas dan tidak mengambil data sendiri kecuali memang menjadi container.
- Gunakan server component untuk data awal bila memungkinkan; gunakan client component hanya saat butuh interaksi browser.
- Validasi data input form menggunakan schema yang eksplisit, misalnya Zod.
- Jangan duplikasi format harga, tanggal, atau URL builder. Pakai helper di src/lib.

4.3 Styling dan UI

Area	Standar
Tailwind CSS	Class utility boleh digunakan langsung, tetapi pola yang sering dipakai perlu diekstrak ke komponen.
Responsive	Setiap halaman public dan admin harus aman di desktop dan mobile. Teks tidak boleh saling menimpa.
Icon	Gunakan icon library yang sudah tersedia untuk aksi UI. Jangan membuat SVG manual jika icon tersedia.
Theme	Komponen harus tetap terbaca di light dan dark mode bila halaman mendukung theme switch.
Image produk	Foto produk harus berasal dari product.imageUrl. Jika kosong, gunakan placeholder netral, bukan gambar produk palsu.
i18n	Copy yang tampil ke user sebaiknya masuk messages i18n bila dipakai di halaman public.

4.4 Data Flow Frontend

Kebutuhan	Standar Implementasi
Fetch data public	Gunakan API wrapper pusat agar error handling dan base URL konsisten.
Admin session	Akses halaman admin harus melalui helper session dan backend cookie auth.
Stock update	Gunakan SSE bila tersedia dan fallback snapshot/polling saat stream gagal.
CTA eksternal	Tokopedia dan WhatsApp diperlakukan sebagai kanal eksternal, bukan checkout internal.
Empty state	Tampilkan pesan kosong yang jelas saat produk/kategori tidak tersedia.

5. Standar Backend

Backend menggunakan Go, chi router, middleware auth/CORS, service layer, repository layer, dan PostgreSQL. Standar backend menekankan pemisahan handler, business logic, dan akses database.

5.1 Struktur Folder Backend

Folder	Fungsi	Standar
cmd/server	Entry point aplikasi API.	Hanya wiring aplikasi, config, database, dan router.
internal/config	Load environment dan konfigurasi.	Semua env harus punya default aman atau validasi wajib.
internal/handler	HTTP handler dan routing.	Handler fokus pada request parsing, auth context, response.
internal/service	Business logic.	Validasi domain dan transaksi penting diletakkan di service.
internal/repository	Query database.	Repository tidak berisi logika UI atau HTTP response.
internal/model	Domain model dan DTO.	JSON tag harus sesuai kontrak frontend.
migrations	Migrasi database.	Migration production harus incremental dan tidak destruktif tanpa rencana eksplisit.

5.2 Naming dan Go Style

Item	Standar
Package	Gunakan nama singkat, lowercase, dan sesuai domain seperti handler, service, repository.
Struct	Gunakan PascalCase untuk exported struct, misalnya ProductService.
Method	Gunakan nama aktif dan jelas, misalnya AdminUpdate dan FindBySlug.
Context	Fungsi repository/service yang mengakses DB menerima context.Context.
Error	Error dikembalikan ke caller. Jangan panic untuk alur request normal.
Formatting	Semua kode Go harus lolos gofmt sebelum commit.

Pola Handler, Service, Repository

- Handler membaca parameter request, decode JSON, memanggil service, lalu menulis response.
- Service menjalankan validasi bisnis seperti stok, status produk, dan aturan delete.
- Repository menjalankan query SQL dan transaksi database.
- Model/DTO menjaga bentuk data agar contract frontend dan backend tetap stabil.

5.3 API Response dan Error Handling

Kondisi	HTTP Status	Standar Response
Request berhasil	200/201	Kembalikan data JSON sesuai contract. Jangan mengubah nama field tanpa update frontend.
Input invalid	400	Kembalikan pesan error singkat dan aman dibaca user/admin.
Unauthorized	401	Digunakan saat cookie/token tidak valid atau belum login.
Forbidden	403	Digunakan saat role/session tidak punya akses.
Data tidak ditemukan	404	Tidak membocorkan detail internal database.
Server error	500	Log detail di server, response ke client tetap generik.

6. Standar Database dan Migration

Area	Standar
Naming tabel	Gunakan plural snake_case, misalnya products, categories, stock_logs.
Naming kolom	Gunakan snake_case di database dan camelCase di JSON response.
UUID	Gunakan UUID untuk primary key sesuai schema yang berjalan.
Audit field	Gunakan created_at dan updated_at untuk data utama. Gunakan stock_logs untuk audit perubahan stok.
Migration	Migration production harus incremental. Hindari DROP CASCADE kecuali memang ada rencana cleanup yang disetujui.
Index	Tambahkan index untuk kolom filter umum seperti slug, status, category_id, dan product_id.
RLS	RLS dapat diaktifkan untuk table public, tetapi backend owner/admin connection tetap menjadi jalur akses utama.

7. Standar Security

Area	Standar
Environment variable	Nilai secret hanya disimpan di local env atau dashboard deployment. Jangan commit nilai secret.
JWT secret	Harus panjang dan unik per environment.
Cookie auth	Gunakan HttpOnly cookie untuk admin session. Secure cookie harus proxy-aware di production HTTPS.
CORS	Batasi origin ke domain frontend yang valid dan localhost untuk development.
Upload/image	Validasi ukuran dan tipe file bila upload digunakan. Jangan percaya nama file dari user.
Logging	Jangan log password, token, database URL, atau data sensitif.

8. Standar Testing

Testing dilakukan untuk memastikan perubahan tidak merusak alur katalog public, admin inventory, API, dan stock update.

Jenis Test	Target	Command / Cara Verifikasi
Frontend unit test	Komponen, helper, store, dan UI logic.	npm run test
Frontend build	Validasi Next.js build dan TypeScript.	npm run build
Backend unit/API test	Service, repository, handler, dan middleware.	go test ./...
Backend build	Validasi binary backend.	go build -tags netgo -ldflags "-s -w" -o planet-motor-bmw-api ./cmd/server
Manual UI	Homepage, products, product detail, admin CRUD, responsive.	Cek browser desktop dan mobile.
Integration	API products, categories, stock snapshot, admin login/logout.	Cek endpoint dan flow dari frontend.

9. Checklist Code Review

Checklist	Pertanyaan Review
Scope	Apakah perubahan sesuai task dan tidak menyentuh file yang tidak terkait?
Type safety	Apakah type TypeScript dan struct Go sesuai data contract?
Error handling	Apakah error dari API, DB, dan form ditangani dengan benar?
Security	Apakah tidak ada secret, token, password, atau URL sensitif yang ikut ter-commit?
UI	Apakah tampilan responsive dan tidak ada teks/elemen tumpang tindih?
Database	Apakah migration aman untuk data production?
Testing	Apakah test dan build yang relevan sudah dijalankan?
Docs	Apakah README atau dokumen perlu diperbarui karena contract berubah?

10. Definisi Done

Sebuah perubahan dianggap selesai bila:

- Kode mengikuti struktur folder dan naming yang ditetapkan.
- Tidak ada secret atau credential di source code dan dokumentasi.
- Frontend dan backend tetap kompatibel dengan API contract yang aktif.
- Build dan test relevan berhasil dijalankan atau alasan kegagalannya dicatat.
- UI utama sudah dicek pada desktop dan mobile bila perubahan menyentuh tampilan.
- Perubahan database sudah memiliki migration yang aman dan dapat diaudit.

11. Penutup

Standar ini digunakan sebagai acuan bersama dalam implementasi Planet Motor BMW. Jika project berkembang, dokumen ini dapat diperbarui dengan aturan baru, tetapi perubahan standar harus tetap menjaga prinsip utama: konsisten, aman, mudah dibaca, dan mudah diuji.